Performance comparison

storefront · 742d417 → e94a922 · Detailed report · generated 17 Oct 2026, 16:45

BEFORE

742d417

Author	Demo Developer
Date	15 Oct 2026, 10:24
Message	Product list with a live ticker
Branch	main
Scenario	perf-profiler demo
Runs	3 (page load)

AFTER

e94a922

Author	Demo Developer
Date	16 Oct 2026, 16:05
Message	Memoise list rows and move the ticker out of App
Branch	main
Scenario	perf-profiler demo
Runs	3 (page load)

Improved

Key metrics got better and none got significantly worse. Medium confidence: see the comparability notes.

✓ 5 improved **✕ 2 degraded** **= 19 unchanged** **1 not measured**

Highlights

- ✓ Wasted renders: 59 → 0 (–100%)
- ✓ JavaScript CPU time: 1.58 s → 110 ms (–93%)
- ✕ TickHeading (src/main.jsx) — renders: 0 → 60 (new)
- ✓ Row (src/Row.jsx) — renders: 7,200 → 120 (–98%)
- ✓ App (src/main.jsx) — renders: 60 → 1 (–98%)
- ✓ Wasted (src/Wasted.jsx) — renders: 60 → 1 (–98%)

Comparability

- ✓ **Same page.** /
- ✓ **Same mode (page load).**
- ✓ **Same CPU setting (1×).**
- ✓ **3 runs before, 3 after.** Medians are used and differences must exceed the run-to-run spread.
- ! **Different file-attribution modes.** sourcemap + urls / sourcemap: function lines may differ between sides.
- ! **Some recordings were nearly idle.** 3 run(s) executed under 300 ms of JavaScript: little was measured.

All metrics

METRIC	BEFORE	AFTER	CHANGE	Δ%	RESULT
RESPONSIVENESS					
Total blocking time key range 0 ms–0 ms → 0 ms–0 ms	0 ms	0 ms	0 ms (0%)		= Unchanged
Long tasks (> 50 ms) key range 0–0 → 0–0	0	0	0 (0%)		= Unchanged
Longest task range 0 ms–0 ms → 0 ms–0 ms	0 ms	0 ms	0 ms (0%)		= Unchanged
Slowest interaction key	–	–	–		– n/a
LOADING					
Largest contentful paint key range 136 ms–168 ms → 116 ms–124 ms	140 ms	116 ms	–24.0 ms (–17%)		= Unchanged
First contentful paint range 136 ms–168 ms → 116 ms–124 ms	140 ms	116 ms	–24.0 ms (–17%)		= Unchanged
VISUAL STABILITY					
Layout shift (CLS) key range 0.0015–0.0015 → 0.0015–0.0015	0.0015	0.0015	0.000 (0%)		= Unchanged
CPU AND MAIN THREAD					
JavaScript CPU time key range 1.57 s–1.58 s → 108 ms–118 ms	1.58 s	110 ms	–1.47 s (–93%)		✓ Improved
Scripting range 1.50 s–1.50 s → 49.4 ms–50.7 ms	1.50 s	49.8 ms	–1.45 s (–97%)		✓ Improved
Layout range 3.4 ms–4.0 ms → 3.5 ms–3.9 ms	3.4 ms	3.7 ms	+0.29 ms (+8%)		= Unchanged
Style recalculation range 0.26 ms–0.29 ms → 0.29 ms–0.35 ms	0.28 ms	0.30 ms	+0.02 ms (+7%)		= Unchanged
Paint range 9.5 ms–10.4 ms → 10.5 ms–11.0 ms	10.1 ms	10.7 ms	+0.59 ms (+6%)		= Unchanged
Garbage collection range 4.9 ms–7.8 ms → 0.65 ms–0.89 ms	6.2 ms	0.79 ms	–5.4 ms (–87%)		= Unchanged
Script compile / parse range 2.0 ms–2.1 ms → 2.1 ms–2.3 ms	2.1 ms	2.1 ms	+0.01 ms (0%)		= Unchanged
LAYOUT WORK					
Layouts performed range 61–62 → 61–62	61	61	0 (0%)		= Unchanged
Style recalculations range 2–2 → 2–2	2	2	0 (0%)		= Unchanged
Forced reflows key range 0–0 → 0–0	0	0	0 (0%)		= Unchanged
Time in forced reflows range 0 ms–0 ms → 0 ms–0 ms	0 ms	0 ms	0 ms (0%)		= Unchanged

METRIC	BEFORE	AFTER	CHANGE	Δ%	RESULT
REACT					
Component renders range 7,380–7,380 → 183–183	7,380	183	−7,197 (−98%)		✓ Improved
Wasted renders key range 59–59 → 0–0	59	0	−59 (−100%)		✓ Improved
React render time range 1.46 s–1.46 s → 30.5 ms–31.3 ms	1.46 s	30.6 ms	−1.43 s (−98%)		✓ Improved
Slowest React commit range 27.5 ms–28.2 ms → 27.9 ms–28.8 ms	27.8 ms	28.7 ms	+0.90 ms (+3%)		= Unchanged
MEMORY AND DOM					
DOM nodes range 405–1,241 → 1,241–2,063	405	1,652	+1,247 (+308%)		✕ Degraded
Event listeners range 261–792 → 792–1,320	261	1,056	+795 (+305%)		✕ Degraded
JS heap growth key range -6.2 MB–7.9 MB → 1.2 MB–2.1 MB	2.0 MB	1.4 MB	−0.6 MB (−30%)		= Unchanged
NETWORK					
Requests range 15–15 → 15–15	15	15	0 (0%)		= Unchanged
Data transferred range 3.14 MB–3.14 MB → 3.14 MB–3.14 MB	3.14 MB	3.14 MB	+2.1 KB (0%)		= Unchanged

React components

5 compared · 0 unchanged · library code (node_modules) omitted

Degraded

None degraded.

Improved (4)

NAME	WHAT CHANGED
Row `src/Row.jsx:3`	✓ Renders: 7,200 → 120 (–98%) Wasted renders: 0 → 0 ✓ Render time: 1.44 s → 24.6 ms (–98%)
App `src/main.jsx:42`	✓ Renders: 60 → 1 (–98%) Wasted renders: 0 → 0 ✓ Render time: 8.2 ms → 0.80 ms (–90%)
Wasted `src/Wasted.jsx:2`	✓ Renders: 60 → 1 (–98%) ✓ Wasted renders: 59 → 0 (–100%) Render time: 0.10 ms → 0 ms
Panel `src/Panel.jsx:4`	✓ Renders: 60 → 1 (–98%) Wasted renders: 0 → 0 Render time: 0.30 ms → 0.30 ms

Newly active in the “after” recording (new cost) (1)

NAME	WHAT CHANGED
TickHeading `src/main.jsx:34`	✕ Renders: 0 → 60 (new) Wasted renders: 0 → 0 Render time: 0 ms → 1.3 ms

Files

11 compared · 6 unchanged · library code (node_modules) omitted

Degraded

None degraded.

Improved (3)

NAME	WHAT CHANGED
src/Row.jsx	CPU time: 3.7 ms → 0.31 ms Blocking time: 0 ms → 0 ms ✓ React render time: 1.44 s → 24.6 ms (–98%) Forced reflows: 0 → 0
src/heavy.js	✓ CPU time: 1.42 s → 22.9 ms (–98%) Blocking time: 0 ms → 0 ms React render time: 0 ms → 0 ms Forced reflows: 0 → 0
src/main.jsx	CPU time: 0.82 ms → 0.52 ms Blocking time: 0 ms → 0 ms ✓ React render time: 8.2 ms → 2.3 ms (–72%) Forced reflows: 0 → 0

Functions

87 compared · 85 unchanged · library code (node_modules) omitted

Degraded

None degraded.

Improved (1)

NAME	WHAT CHANGED
burn `src/heavy.js:2`	✓ Self time: 1.42 s → 22.9 ms (–98%)

Problems found

New problems introduced (6)

CRITICAL · Network — react-dom-hmN8LOy-.js?… is 2.47 MB over the wire

Fix: Split the bundle, drop unused dependencies, or lazy-load it.

WARNING · Network — react-dom-hmN8LOy-.js?… is served uncompressed (2532 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Network — react.js?… is served uncompressed (217 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Memory — Large DOM: 2,063 nodes

Fix: Virtualise long lists and tables, and unmount hidden tabs/modals instead of hiding them.

WARNING · Network — react_jsx-dev-runtime.js?… is served uncompressed (104 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · React — TickHeading rendered 60× (19/s)

`TickHeading()` in `src/main.jsx:34`

Fix: Find what updates this often (a timer, a store subscription, a scroll handler) and throttle it or narrow the selector/subscription.

Problems resolved (12)

CRITICAL · React — Row is slow to render: 1442ms total, 0.2ms per render (7200×)

`Row()` in `src/Row.jsx:3`

Fix: Move expensive calculations into useMemo, virtualise long lists, and avoid deriving large data structures inside render.

CRITICAL · CPU — src/heavy.js spent 1420ms of JavaScript (90% of CPU)

`burn()` in `src/heavy.js:2`

Fix: Look for work repeated on every render/event, loops over large data, or JSON/regex/sort work that can be cached, memoized or moved to a worker.

CRITICAL · Network — react-dom-hmN8LOy-.js?… is 2.47 MB over the wire

Fix: Split the bundle, drop unused dependencies, or lazy-load it.

WARNING · React — Row gets a new function for “onPick” on every render (7080×)

`Row()` in `src/Row.jsx:3`

Fix: Create “onPick” with useCallback (or hoist it out of the parent).

WARNING · Network — react-dom-hmN8LOy-.js?… is served uncompressed (2532 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Network — react.js?… is served uncompressed (217 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · Network — react_jsx-dev-runtime.js?… is served uncompressed (104 KB)

Fix: Enable gzip/brotli for JS, CSS and JSON responses.

WARNING · React — App rendered 60× (19/s)

`App()` in `src/main.jsx:15`

Fix: Find what updates this often (a timer, a store subscription, a scroll handler) and throttle it or narrow the selector/subscription.

WARNING · React — Panel gets a new function for “onSelect” on every render (59×)

`Panel()` in `src/Panel.jsx:4`

Fix: Create “onSelect” with useCallback (or hoist it out of the parent).

WARNING · React — Panel gets a new object for “config” on every render (59×)

`Panel()` in `src/Panel.jsx:4`

Fix: Create “config” with useMemo, or hoist the constant out of the component.

WARNING · React — Panel rendered 60× (19/s)

`Panel()` in `src/Panel.jsx:4`

Fix: Find what updates this often (a timer, a store subscription, a scroll handler) and throttle it or narrow the selector/subscription.

WARNING · React — Wasted re-rendered 60× — 59 with nothing changed (98%)

`Wasted()` in `src/Wasted.jsx:2`

Fix: Wrap in React.memo, or stop the parent re-rendering (move state down, split the component).

Problems that persist (1)

ISSUE	IMPACT BEFORE	IMPACT AFTER
warning · Row rendered 120× (39/s)	1.44 s	25.0 ms

Recordings used

SIDE	RECORDING	COMMIT	MODE	LENGTH	CPU	URL
Before	20261017T111444-load	742d417	page load	3.13 s	1×	http://localhost:5236/
Before	20261017T111448-load	742d417	page load	3.15 s	1×	http://localhost:5236/
Before	20261017T111452-load	742d417	page load	3.14 s	1×	http://localhost:5236/
After	20261017T111459-load	e94a922	page load	3.11 s	1×	http://localhost:5236/
After	20261017T111503-load	e94a922	page load	3.11 s	1×	http://localhost:5236/
After	20261017T111507-load	e94a922	page load	3.12 s	1×	http://localhost:5236/

Methodology

What is compared. Profiler reports recorded in Chrome through the DevTools Protocol (tracing, V8 CPU sampling and an in-page React probe). When a side has several runs, each metric is the **median** of its runs.

What counts as a change. A metric is “improved” or “degraded” only when the difference exceeds **all three** of: its absolute floor, its relative floor (both in the table below) and the **run-to-run spread** seen on either side. Everything else is “unchanged”. For every metric listed here **lower is better**.

Overall verdict. Decided only by the *key* metrics: improved if some improved and none degraded; regressed if some degraded and none improved; mixed if both; otherwise no significant change.

Limits. A single run per side cannot separate a real change from noise: record each side at least three times, on the same page, in the same mode, with the same CPU setting. Interaction recordings are only comparable if the same actions were performed. Dev builds are slower than production: trust direction and ranking more than absolute numbers.

METRIC	GROUP	ABS. FLOOR	REL. FLOOR	KEY
Total blocking time	Responsiveness	50.0 ms	10%	yes
Long tasks (> 50 ms)	Responsiveness	1	15%	yes
Longest task	Responsiveness	25.0 ms	10%	
Slowest interaction	Responsiveness	40.0 ms	10%	yes
Largest contentful paint	Loading	100 ms	5%	yes
First contentful paint	Loading	50.0 ms	5%	
Layout shift (CLS)	Visual stability	0.010	20%	yes
JavaScript CPU time	CPU and main thread	50.0 ms	10%	yes
Scripting	CPU and main thread	50.0 ms	10%	
Layout	CPU and main thread	20.0 ms	15%	
Style recalculation	CPU and main thread	20.0 ms	15%	
Paint	CPU and main thread	20.0 ms	15%	
Garbage collection	CPU and main thread	20.0 ms	20%	

METRIC	GROUP	ABS. FLOOR	REL. FLOOR	KEY
Script compile / parse	CPU and main thread	20.0 ms	20%	
Layouts performed	Layout work	10	15%	
Style recalculations	Layout work	10	15%	
Forced reflows	Layout work	5	20%	yes
Time in forced reflows	Layout work	10.0 ms	20%	
Component renders	React	50	15%	
Wasted renders	React	25	15%	yes
React render time	React	25.0 ms	10%	
Slowest React commit	React	10.0 ms	15%	
DOM nodes	Memory and DOM	100	5%	
Event listeners	Memory and DOM	50	10%	
JS heap growth	Memory and DOM	5.0 MB	15%	yes
Requests	Network	3	10%	
Data transferred	Network	100.0 KB	5%	